Week 4 Coding Challenge Summary

Nov 7, 2026 · @Robert

This week I turned the plan into a working app. I focused on UI and UX design first, had AI reviewers check the designs, built the ADHD Financial App, tested it on my Android phone, and connected it to a real database with accounts.

Links

- **GitHub repository:** github.com/willsrobjr124/AI-Development-Residency. This week's work is in the Week 4 Coding Challenge folder, and the app code is in `ADHDbudget-App`.
- **Live app on Vercel:** aidevelopmentresidency.vercel.app.

Where things stand

Part of the work	Status
Design system and 76 Figma screens	Done, and more polish is planned
The app and its 7 core features	Built and working on my phone
Accounts and Supabase database	Built and working: I created an account on my phone
Public link on Vercel	Live at aidevelopmentresidency.vercel.app

1. Setting up for design work

- Created the Week 4 project folder, with a guide file (`CLAUDE.md`) that tells Claude Code the scope, the design process and the rules.
- Grew the skill library to 27 skills. A skill is like a recipe card that the AI reads before it builds each part.
- Added 5 new design skills: design brief, information architecture, design tokens, design review and mobile app UI design.

- Reviewed each new skill for safety before installing it, and decided which skill wins when two disagree. Our ADHD and accessibility rules always come first.
- Created 3 subagents, explained in the next section.

2. The AI agents I created

A subagent is an AI helper with one job and its own instructions. Mine read the project and give advice, but they don't change the code, so the decisions stay with me.

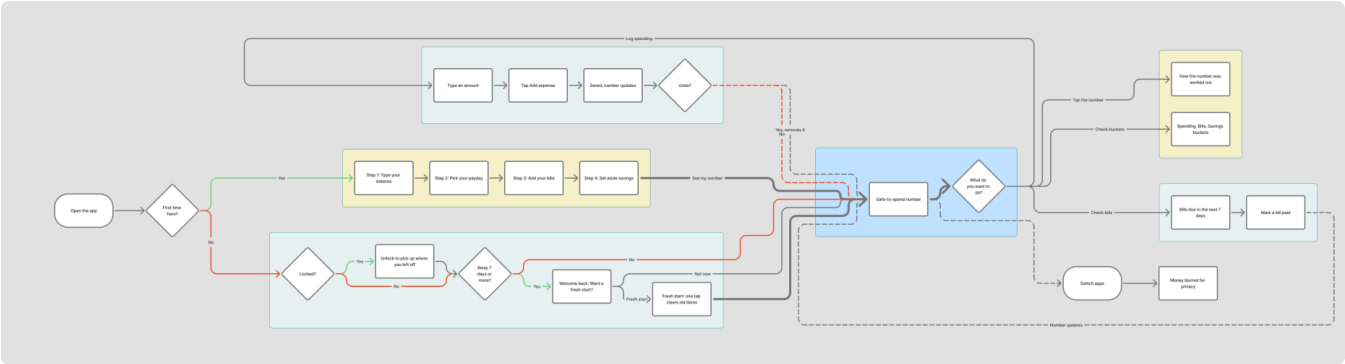
Agent	Its job	How I used it
Engineer	Checks if an idea can be built, how hard it is, and what could go wrong	Reviewed 47 screens, ranked features by how quickly they could be built, and suggested calm motion ideas
Accessibility reviewer	Critiques designs against WCAG 2.2 AA, plus our rules for tap sizes and text	Reviewed 44 screens, calculated real color contrast, and wrote 25 notes
Strategic communications specialist	Turns technical work into clear summaries and business cases for leaders	Ready for updates to my instructor and stakeholders. I haven't needed it yet

Each agent looks at the work from one point of view, so it catches things I would miss. Each also keeps its notes in its own document, which gives me a record of what was found and what changed.

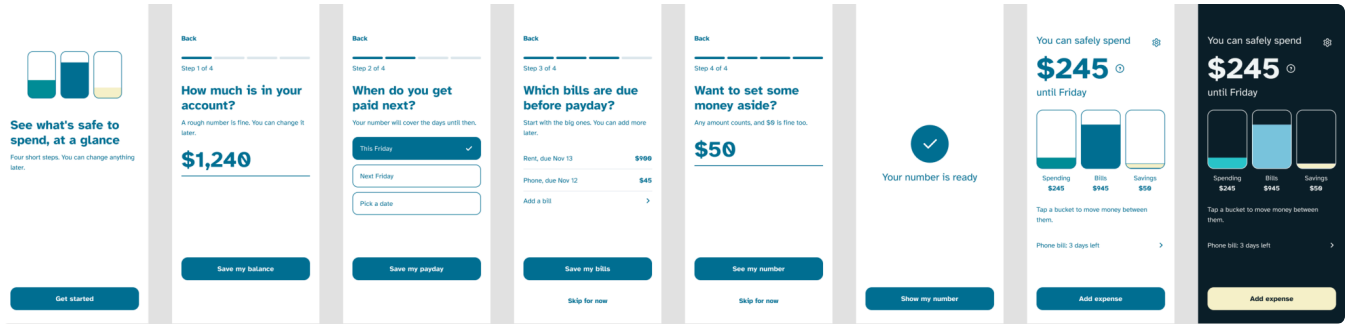
3. Creative direction and design

- Mapped the full user journey as a flow chart in FigJam.
- Studied design references. I liked the simple Loowner onboarding, because few distractions suit people with ADHD. Our screens are original.
- Wrote a design brief with a calm, minimal style: white backgrounds, deep blue text, the Atkinson Hyperlegible font and a kind-friend voice.
- Designed 76 screens in Figma, including onboarding, home, every feature flow, the Epic 1 and 2 features, and accounts.

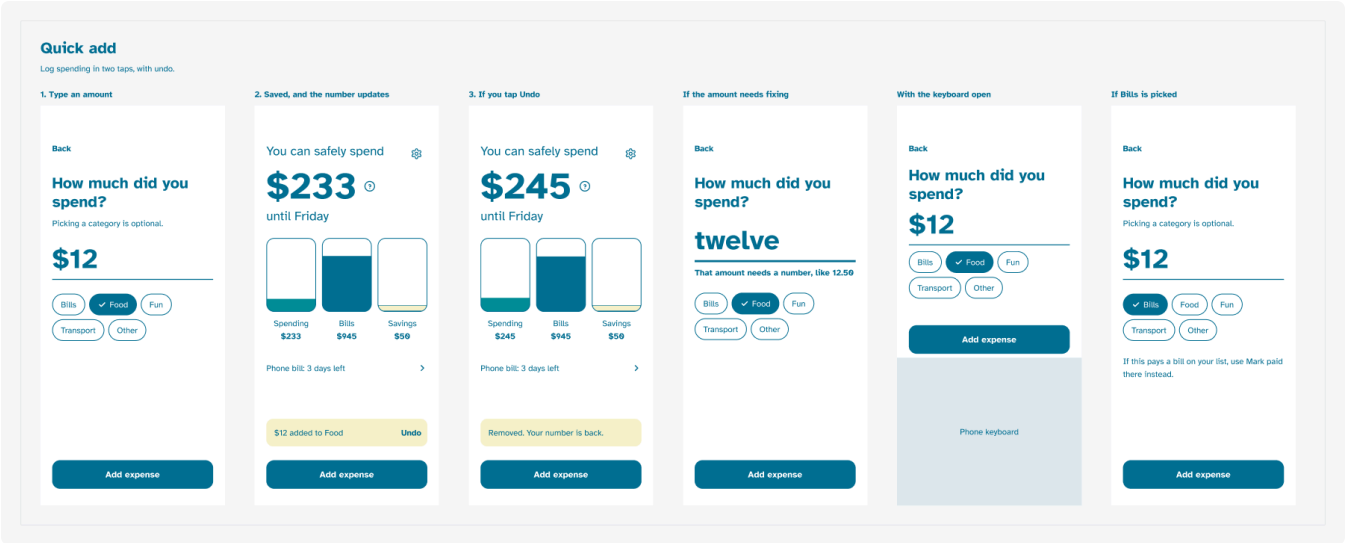
- Built a small design system in Figma: shared colors, text styles and reusable parts like buttons, chips, buckets and a calendar.



The user journey I mapped in FigJam before designing any screens. The plan changed later: a solid cover replaced the blur.



Onboarding and home in Figma, with one idea on each screen.

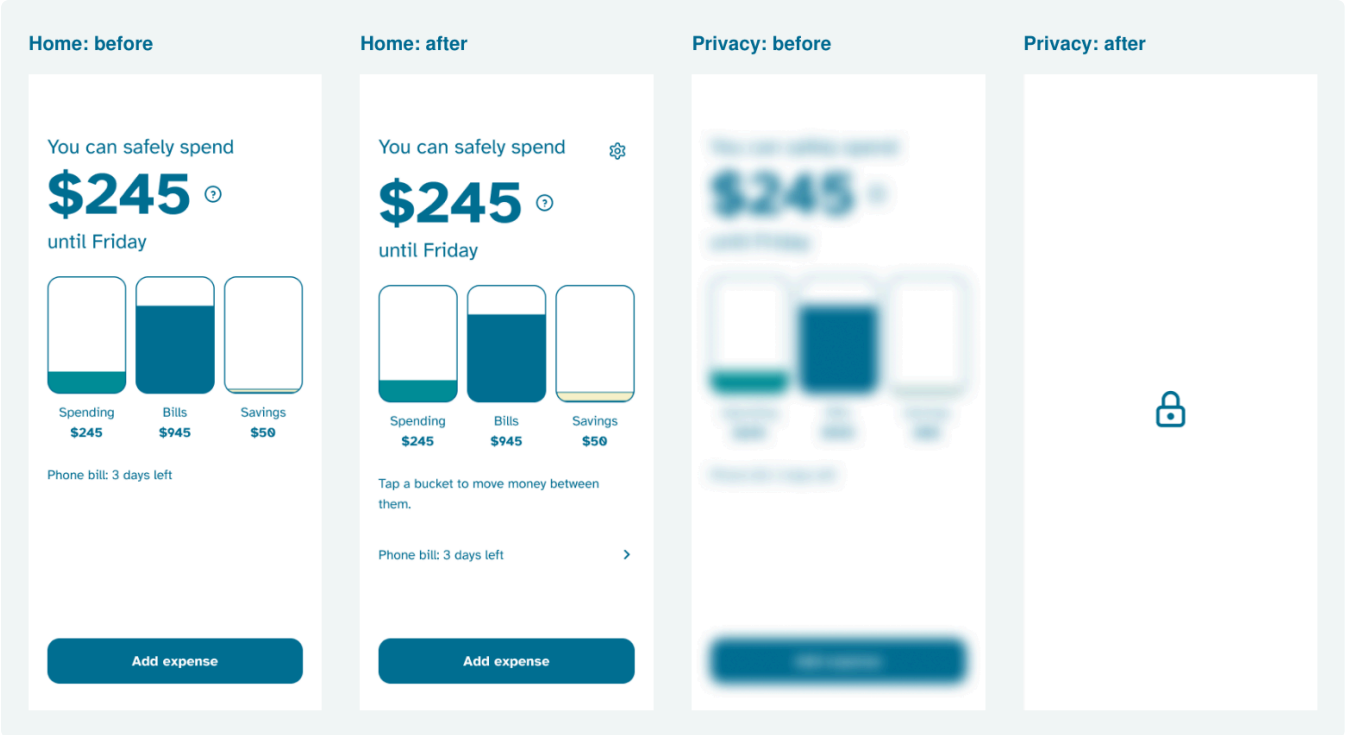


Each feature was designed as a full flow, including errors and the keyboard.

4. AI design reviews

I asked two subagents to critique the designs and write their notes in separate documents.

Reviewer	What it found	What changed
Accessibility reviewer	25 notes against WCAG 2.2 AA, including 2 blockers: home didn't work with large text, and the app was locked to portrait	Fixed or partly fixed 20 notes in Figma, and later fixed the screen rotation setting
Engineer	All 7 Must-haves could be built without a backend. The biggest gap was that the number went wrong after payday	Added a payday prompt, a fix for Mark paid, a monthly bill option and a breakdown that adds up



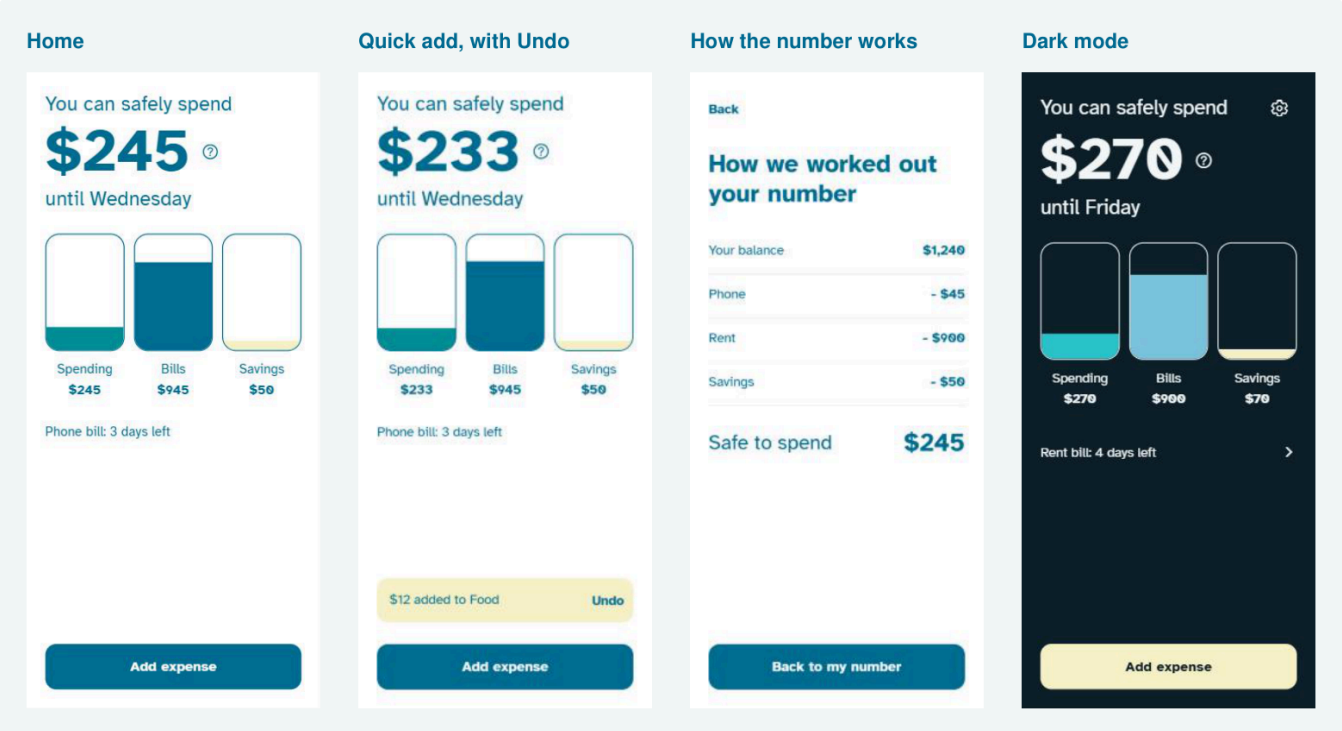
What the reviews changed. Home gained a tappable bill row, a helper line, a settings gear and a clearer Savings level. The blur left "$245" readable, so a solid cover replaced it.

5. The app I built

The app is built with Expo, so one set of code runs on Android and on the web. It has 28 screens.

- **Quick setup:** four short steps that save your place.
- **Safe-to-spend number:** the first thing on home, with a ? that shows how it was worked out.
- **Buckets:** Spending, Bills and Savings. Tap one to move money or see bills.
- **Quick add with undo:** log an expense in a couple of taps.
- **Bill countdown:** see what's due, mark bills paid, and add new ones with a calendar.
- **Fresh start:** after a week away, confirm your balance with no guilt.
- **Privacy:** a solid cover hides money in the app switcher, and the app locks after 5 idle minutes.
- **Also added:** Recent spending, Settings, dark mode, large-text layouts and a three-jar app icon.

Motion stays calm and short: gentle press feedback, confirmations that fade in, and bucket levels that slide. It all switches off with Reduce motion. I left out confetti, bounce and streaks on purpose, because the ADHD rules say to.

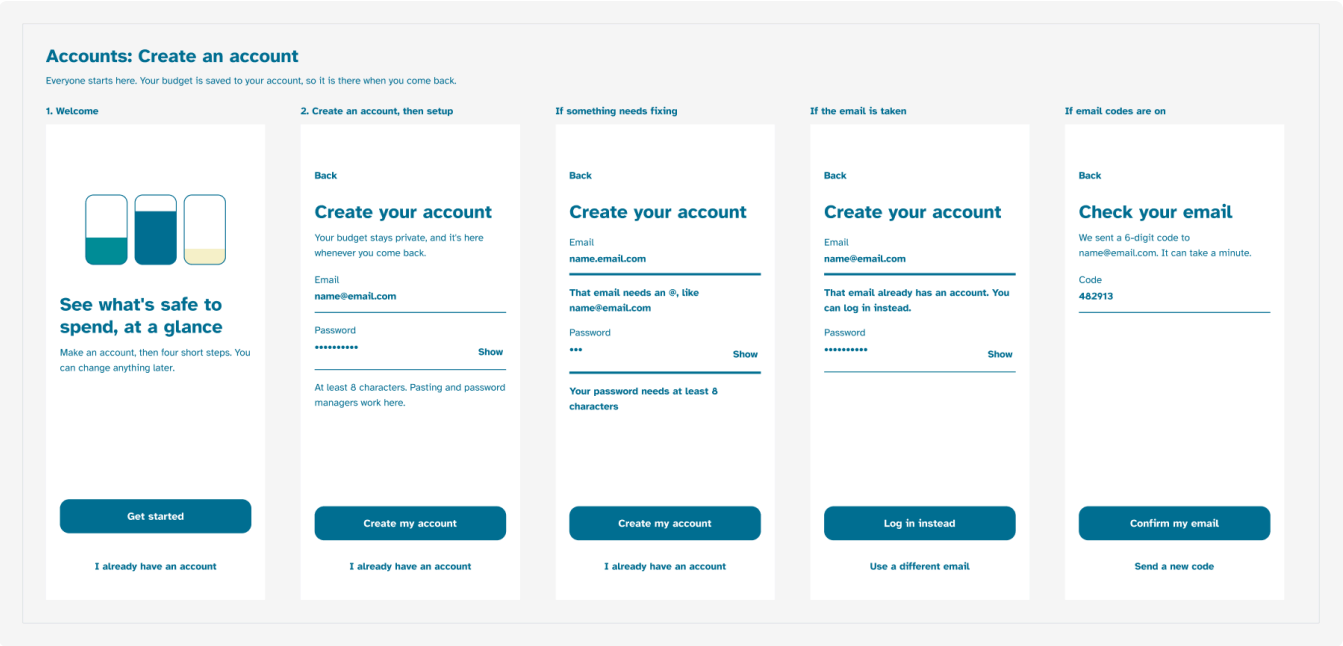


The app running in the web preview during testing, using test numbers.

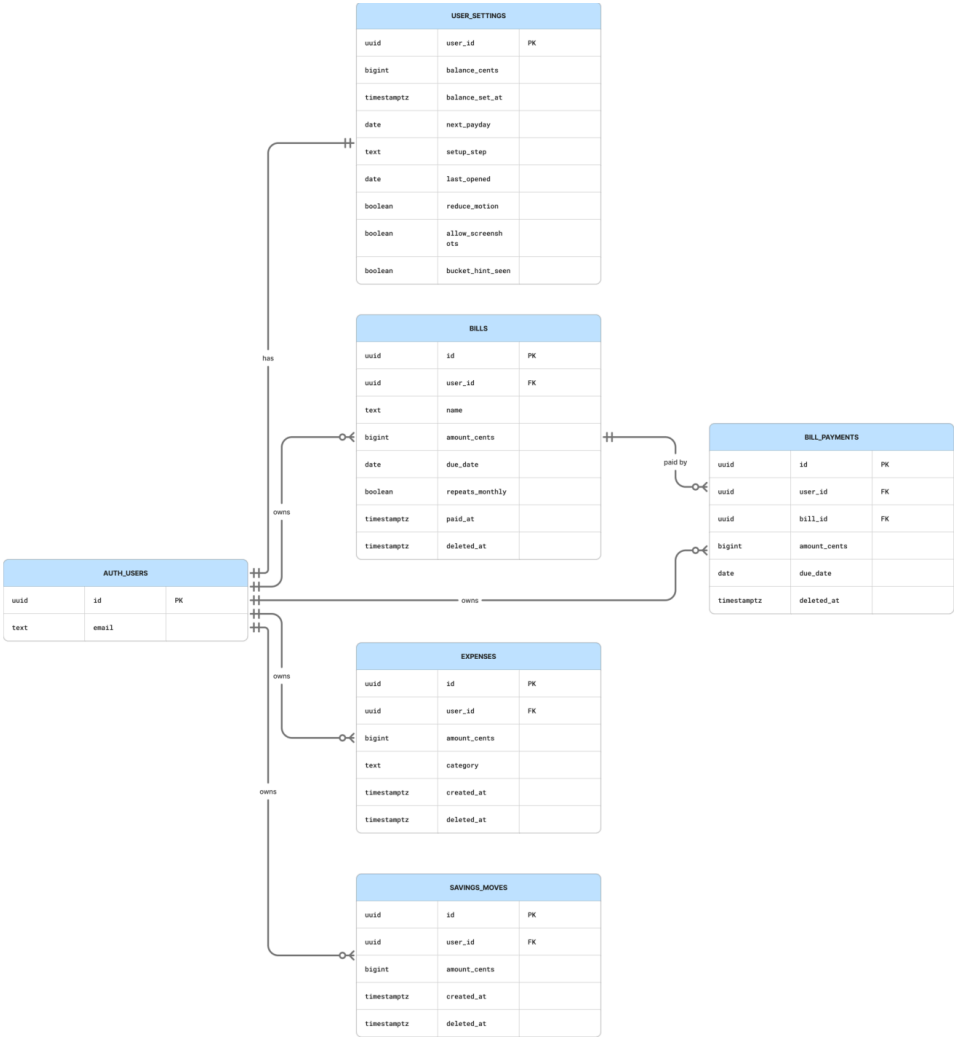
6. Accounts and database

My instructor's requirements called for accounts and a real database, so I added them this week.

- **Supabase database:** 5 tables hold settings, bills, payments, expenses and savings.
- **Accounts:** create account, log in, log out, reset password and delete account.
- **Privacy between people:** a test with two made-up people showed that one could not see or change the other's data. Logged-out visitors were blocked, and Supabase's security check found 0 problems.
- **Safe saving:** double taps count once, and a failed save leaves nothing half-done.



The create account flow in Figma, designed before it was built.



The database diagram. Every table belongs to one person.

7. Testing

- 96 automated tests pass, along with type checking, lint and Expo's 20 project checks.
- I tested onboarding and adding bills on my own Android phone.
- Walking through the app by hand caught real bugs, like a dead-end page during setup.

8. Backlog and decisions

- Updated the Kanban board so MVP #1 matches what exists now, plus accounts, polish and publishing. MVP #1 has 14 tickets.
- Moved 10 tickets to a new MVP #2, including the goal picker, bank linking, the widget, reminders and Focus Mode.
- Rewrote the security ticket to match what was really built.

- Wrote 6 Architecture Decision Records (ADRs), which explain the big choices and why I made them. I also made a database diagram.

One decision to confirm: the requirements ask for Next.js with Material UI. I built an Expo app instead, and that is recorded in ADR 1. Expo can still publish to the web on Vercel. I am confirming this with my instructor.

What I learned

- **Check the requirements early.** The full list arrived late, and it changed the database, login and hosting plans.
- **Review before building.** The AI reviewers found problems in the designs, where they were cheap to fix.
- **Test by using it.** Automated tests passed while real problems were still there.
- **Design rules need an order.** When advice clashed, putting ADHD and accessibility first made decisions easy.

What's next

- Finish testing accounts and saving on my phone.
- Make an Expo Go link that works on any phone, without my computer running.
- Make an Android install link.
- Redesign the screens in Figma, then build the changes into the app.
- Have the accessibility reviewer re-check the finished app.
- Move finished tickets to Done.

Sources

- W3C WCAG 2.2, plus the Expo SDK 58 and Supabase documentation.
- Spiel, K., Hornecker, E., Williams, R. M., & Good, J. (2022). ADHD and technology research: Investigated by neurodivergent readers. CHI 2022.
- Bangma, D. F., Tucha, L., Fuermaier, A. B. M., Tucha, O., & Koerts, J. (2020). Financial decision-making in a community sample of adults with and without current symptoms of ADHD. PLOS ONE, 15(10), e0239343.
- ADHD UX guides (2025 to 2026): Accessibility Checker, UXPA, Din Studio, Design Monks, iFactory. Also our own design, review and requirements documents.